

Peregrine C++ Web Framework — Complete Technical Documentation

- **Framework Name:** Peregrine Web Framework (`peregrine_framework` / `peregrine::framework`)
 - **Version:** 1.0.0
 - **Primary Language:** C++
 - **Architecture:** Modular, Header-Only, Socket/TLS Native Framework
 - **Target Audience:** Software Engineers, Backend Architects, and Embedded/Systems Developers integrating high-performance web APIs, microservices, Single Page Applications (SPA), or real-time streaming services.
-

Table of Contents

1. [Framework Overview](#)
2. [Core Concepts](#)
3. [Architecture & Request Lifecycle](#)
4. [Installation & Build System](#)
5. [Project Structure](#)
6. [Quick Start Guide](#)
7. [Configuration System](#)
8. [API Reference](#)
9. [Routing Engine](#)
10. [Request & Response Handling](#)
11. [Middleware & Request Hooks](#)
12. [Authentication & Session Management](#)
13. [Database Integration Patterns](#)
14. [External API Integration](#)
15. [Error Handling & Exceptions](#)
16. [Data Validation](#)
17. [Security Architecture](#)
18. [Testing & Quality Assurance](#)
19. [Logging & Monitoring](#)
20. [Performance & Concurrency Considerations](#)
21. [Production Deployment](#)
22. [Integration Guide \(Existing C++ Codebases\)](#)
23. [Extension & Plugin Development](#)
24. [Migration Guide](#)
25. [Common Use Cases](#)
26. [Troubleshooting Guide](#)
27. [Frequently Asked Questions \(FAQ\)](#)

- 28. [Best Practices](#)
 - 29. [Anti-Patterns](#)
 - 30. [Compatibility Matrix](#)
 - 31. [Complete Reference Application](#)
 - 32. [Developer Workflow](#)
 - 33. [Glossary](#)
 - 34. [Documentation Standards & Verification](#)
 - 35. [Source Verification](#)
 - 36. [Final Integration Checklist](#)
-

1. Framework Overview

What It Is

The **Peregrine C++ Web Framework** is a lightweight, high-performance, header-only web framework written in modern C17. It ports the declarative elegance, modular architecture, and developer ergonomics of Python's [Pallets Flask](#) directly into native C.

Problem It Solves

Traditional C++ web development often requires massive heavyweight frameworks (such as Boost.Beast, Poco, or Drogon) with steep learning curves, cumbersome boilerplate, and complex dependency graphs. Peregrine provides an intuitive, zero-overhead alternative built directly on standard POSIX sockets and OpenSSL, allowing C++ developers to write clean, expressive web services and APIs with zero third-party web runtime dependencies.

Primary Use Cases

- High-performance RESTful JSON microservices.
- Embedded web servers (e.g., IoT devices, IP cameras, edge appliances).
- Serving modern Single Page Applications (React, Vue, Vite, Angular).
- Real-time video/audio streaming (MJPEG camera feeds, Server-Sent Events).
- Native C++ backend microservices integrating with existing C/C++ libraries.

Who Should Use It

- C++ developers wanting Flask/Express-like expressiveness without sacrificing native execution speed.
- Systems and embedded engineers requiring a minimal web layer without multi-gigabyte dependency trees.
- Teams building high-throughput services with deterministic memory usage.

Who Should NOT Use It

- Massive enterprise web applications needing distributed clustering, distributed transaction managers, or full WSGI/ASGI Python ecosystems.
- Environments lacking basic POSIX socket or OpenSSL support.

Major Features

- **Header-Only Library:** Include `<peregrine/peregrine.hpp>` and build immediately.
 - **Expressive Routing:** Dynamic converters (`<int:id>` , `<string:name>` , `<path:filepath>`) and automatic RFC-compliant 405 Method Not Allowed .
 - **First-Class Blueprints:** Modular route groups with URL prefixes and scoped hooks.
 - **RESTful Class-Based Views:** `peregrine::MethodView` class polymorphism.
 - **Stateless HMAC-SHA256 Sessions:** Tamper-proof signed client cookies mirroring `itsdangerous` .
 - **CSRF Defense:** Automatic protection for state-mutating requests (`POST` , `PUT` , `PATCH` , `DELETE`).
 - **Live Streaming & MJPEG:** Native chunked and multipart video stream generation (`Response::mjpeg`).
 - **Directory Traversal Protection:** CWE-22 and null-byte injection guards built into static file serving.
 - **Unified HTTP/HTTPS Engine:** Switch between plain HTTP and TLS 1.2/1.3 with a single CLI configuration.
 - **Modern Build Integration:** First-class CMake target export (`peregrine::framework`) and Makefile support.
-

2. Core Concepts

Application (`peregrine::App` / `peregrine::Flask`)

- **Definition:** The central coordination instance of the framework.
- **Purpose:** Holds routes, Blueprints, middleware hooks, configuration dictionaries, and the socket listener loop.
- **Usage:** Instantiate once in your application entrypoint (`main.cpp`).

Router & Routes (`peregrine::Router` , `peregrine::Route`)

- **Definition:** The pattern-matching engine that directs incoming HTTP requests to handlers based on HTTP method and URL path.
- **How it Works:** Compiles path patterns into regular expressions and extracts dynamic parameters into `req.path_params` .
- **Example:** `app.get("/users/<int:user_id>", get_user_handler);`

Handlers (`peregrine::Handler`)

- **Definition:** A callable with the signature `std::function<peregrine::Response(peregrine::Request&)>` .
- **Purpose:** Executes business logic for an incoming request and returns a structured `Response` .

Blueprints (`peregrine::Blueprint`)

- **Definition:** A self-contained collection of routes, scoped middleware hooks, and error handlers.
- **Purpose:** Enables modular application architecture by isolating features (e.g. `auth_bp` , `users_bp` , `products_bp`) with shared URL prefixes.
- **When to Use:** Any project with more than 3 endpoints or multi-developer feature branches.

Class-Based Views (`peregrine::MethodView`)

- **Definition:** An object-oriented dispatch controller where HTTP verbs (`GET` , `POST` , `PUT` , `DELETE`) map to virtual member functions.
- **When to Use:** Clean RESTful CRUD endpoints.

Request (`peregrine::Request`)

- **Definition:** The data structure representing the incoming HTTP request.
- **Properties:** `method` , `path` , `query` (args), `headers` , `cookies` , `form` , `files` , `body` , `session` , `g` , `scheme` , `remote_addr` .

Response (`peregrine::Response`)

- **Definition:** The data structure representing the HTTP response returned to the client.
- **Constructors:** `Response::text()` , `Response::html()` , `Response::json()` , `Response::redirect()` , `Response::stream()` , `Response::mjpeg()` .

Signed Sessions (`peregrine::sign_session` , `peregrine::verify_and_load_session`)

- **Definition:** Stateless client-side session store signed with the application's `secret_key` .
- **Structure:** `base64url(json)` . `base64url(timestamp)` . `base64url(hmac_sha256)` .
- **When to Use:** Authenticated user identity, CSRF tokens, temporary UI state.
- **When NOT to Use:** Large data blobs (>4KB) or unencrypted raw sensitive passwords.

3. Architecture

High-Level Architectural Diagram



+-----+

+-----+

| HTTP/1.1 Parser & Multipart Engine |

+-----+

+-----+

| Signed Session Verification |

+-----+

+-----+

| Global before_request Middleware |

+-----+

+-----+

| Router & Path Converter Engine |

+-----+

+-----+

| Function Route Handler | | MethodView Controller |

+-----+

+-----+

+-----+

+-----+

| Global after_request Middleware |

+-----+

+-----+

| Session Signature & Set-Cookie |

+-----+

+-----+

| HTTP Response Serializer / Stream |

+-----+

Module Responsibilities

Header File	Module	Responsibility
include/peregrine/peregrine.hpp	Master Header	Aggregates all headers; provides namespace peregrine , namespace fw , and namespace flask .
include/peregrine/app.hpp	Core Application	App / Flask coordinator, route dispatching, blueprint wiring, static/SPA engine.
include/peregrine/blueprint.hpp	Blueprints	Modular route groups, prefix scoping, scoped middleware hooks.
include/peregrine/view.hpp	Class-Based Views	View base class and MethodView RESTful class polymorphism.
include/peregrine/routing.hpp	Routing Engine	Regex pattern compilation, parameter extraction, RFC 405 matching.
include/peregrine/request.hpp	Request Wrapper	Request metadata, header lookup, URL query maps, multipart uploads.
include/peregrine/response.hpp	Response Wrapper	HTTP response factories, cookie setters, live stream attachments.
include/peregrine/session.hpp	Signed Sessions	HMAC-SHA256 stateless session encoding, timestamping, verification.
include/peregrine/csrf.hpp	CSRF Protection	Per-session random hex token generation, timing-safe verification.
include/peregrine/template.hpp	Templating	Jinja/Mustache scalar variable substitution and list iteration loops.
include/peregrine/helpers.hpp	Helper Utilities	jsonify , redirect , abort , send_file , send_from_directory .
include/peregrine/config.hpp	Configuration	Key-value store with default settings and from_env() parser.
include/peregrine/connection.hpp	Socket Abstraction	Uniform read/write primitives for raw sockets and OpenSSL SSL* .
include/peregrine/server.hpp	HTTP Engine	HTTP/1.1 streaming body parser, multipart boundary parser, serializer.
include/peregrine/types.hpp	Server Types	ServerConfig , UploadedFile , MIME type resolver.
include/peregrine/json.hpp	JSON Engine	Dynamic JSON value container, recursive-descent parser, serializer.

include/peregrine/common.hpp	Utility Primitives	String manipulation, URL encoding, base64url, HMAC-SHA256, constant-time equal.
------------------------------	-----------------------	---

4. Installation

System Requirements

- **Operating System:** Linux (Ubuntu 18.04+, Debian 10+, RHEL/CentOS 8+, Arch Linux, Alpine), macOS (10.15+), or POSIX-compliant environment.
- **Compiler:** GCC >= 7.0 or Clang >= 5.0 (supporting ISO C++17).
- **Libraries:** OpenSSL (libssl-dev / openssl-devel), POSIX Threads (pthread).
- **Build Tools:** CMake >= 3.14 or GNU Make.

Package Installation Commands

On Debian / Ubuntu:

```
sudo apt-get update
sudo apt-get install -y build-essential libssl-dev cmake
```

On Fedora / RHEL / Rocky Linux:

```
sudo dnf install -y gcc-c++ make cmake openssl-devel
```

On Arch Linux:

```
sudo pacman -S base-devel cmake openssl
```

Adding to an Existing Project

Method 1: CMake Subdirectory (Recommended)

Add this repository to your project (or as a Git Submodule under extern/peregrine), then in your CMakeLists.txt :

```
cmake_minimum_required(VERSION 3.14)
project(MyService LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

```
# Add the framework
add_subdirectory(extern/peregrine)

# Link to your executable
add_executable(my_service src/main.cpp)
target_link_libraries(my_service PRIVATE peregrine::framework)
```

Method 2: Header-Only Direct Include

Copy the `include/peregrine/` directory into your project's include path and link with `OpenSSL` and `pthread` :

```
g++ -std=c++17 src/main.cpp -linclude -lssl -lcrypto -pthread -o my_service
```

5. Project Structure

A clean, production-grade service repository using the framework:

```
my_service/
  CMakeLists.txt      # CMake configuration linking peregrine::framework
  Makefile            # Optional convenience makefile
  include/
    peregrine/        # Peregrine Framework header files
  src/
    main.cpp          # Application entrypoint & CLI argument parsing
    config.hpp         # Service configuration & environment loading
    db.hpp / db.cpp    # Database connection pool / Repository
    services/         # Modular service Blueprints
      auth_service.hpp / .cpp
      user_service.hpp / .cpp
      camera_service.hpp / .cpp
  dist/               # Optional Single Page Application build (React/Vue)
    index.html
    assets/
  tests/              # Unit & integration test suites
    test_api.cpp
  cert.pem / key.pem   # Development TLS certificates
```

6. Quick Start Guide

Complete Minimal Working Example (`main.cpp`)


```

#include <peregrine/peregrine.hpp>
#include <iostream>

int main() {
    // 1. Create the application instance
    peregrine::App app;
    app.secret_key = "development-secret-key-change-in-production";

    // 2. Define a plain text route
    app.get("/", [](peregrine::Request& req) {
        return peregrine::Response::text("Welcome to Peregrine C++ Web Framework!");
    });

    // 3. Define a JSON API route with path parameter
    app.get("/api/greet/<name>", [](peregrine::Request& req) {
        std::string name = req.path_params["name"];

        peregrine::Json response = peregrine::Json::object();
        response["message"] = "Hello, " + name + "!";
        response["success"] = true;

        return peregrine::jsonify(response);
    });

    // 4. Define a POST route accepting JSON body
    app.post("/api/echo", [](peregrine::Request& req) {
        peregrine::Json data = req.get_json();
        if (data.is_null()) {
            return peregrine::Response::json(peregrine::Json::object(), 400);
        }
        return peregrine::jsonify(data, 201);
    });

    // 5. Run server on 0.0.0.0:5000
    std::cout << "Starting server on http://localhost:5000\n";
    app.run("0.0.0.0", 5000);
    return 0;
}

```

Compiling and Testing

```

g++ -std=c++17 main.cpp -linclude -lssl -lcrypto -pthread -o quickstart
./quickstart

```

7. Configuration System

Configuration Object (`peregrine::Config`)

The framework includes a configuration container [`peregrine::Config`] (file:///home/mantra/Documents/HARSHAD/TESTING_PROJECT/frameworkCPP/include/peregrine/config.hpp) that manages environment variables, secrets, and operational constants.

```
peregrine::App app;

// 1. Setting and Getting Configurations
app.config.set("DATABASE_URL", "sqlite:///data.db");
app.config.set("MAX_CONTENT_LENGTH", "33554432"); // 32 MB

std::string db_url = app.config.get("DATABASE_URL");
int max_bytes = app.config.get_int("MAX_CONTENT_LENGTH", 16777216);
bool debug = app.config.get_bool("DEBUG_MODE", false);

// 2. Loading from Environment Variables
app.config.from_env("SECRET_KEY", "APP_SECRET_KEY");
app.config.from_env("PORT", "PORT");
```

8. API Reference

`peregrine::App` / `peregrine::Flask`

Constructors

```
App();
App(std::string name, std::string static_folder = "", std::string static_url_path = "");
```

Route Registration Methods

```
void get(const std::string& path, Handler handler);
void post(const std::string& path, Handler handler);
void put(const std::string& path, Handler handler);
void patch(const std::string& path, Handler handler);
void del(const std::string& path, Handler handler);
void delete_(const std::string& path, Handler handler);
void options(const std::string& path, Handler handler);
void route(const std::string& path, const std::vector<std::string>& methods, Handler handler);
void add_url_rule(const std::string& path, const std::vector<std::string>& methods, Handler handler);
```

Blueprint & Static Serving Methods

```
void register_blueprint(const Blueprint& bp, const std::string& url_prefix = "");
void static_dir(const std::string& url_prefix, const std::string& directory_path);
void serve_spa(const std::string& dist_directory = "dist",
               const std::string& index_file = "index.html",
               const std::string& api_prefix = "api");
void enable_csrf_protection(std::vector<std::string> exempt_paths = {});
```

Middleware & Hooks

```
void before_request(std::function<void(Request&)> fn);
void after_request(std::function<void(Request&, Response&)> fn);
void error_handler(int status_code, std::function<Response(Request&)> fn);
```

Server Runner

```
void run(const std::string& host, int port);
void run(const ServerConfig& config);
```

9. Routing Engine

Path Converters

```
// 1. Integer converter (<int:name>) -> matches digits [0-9]+
app.get("/orders/<int:order_id>", [](peregrine::Request& req) {
    int order_id = std::stoi(req.path_params["order_id"]);
    return peregrine::Response::text("Order ID: " + std::to_string(order_id));
});

// 2. String converter (<string:name> or <name>) -> matches segment [^/]+
app.get("/users/<username>", [](peregrine::Request& req) {
    std::string user = req.path_params["username"];
    return peregrine::Response::text("User: " + user);
});

// 3. Path converter (<path:name>) -> matches entire remaining path including slashes
app.get("/files/<path:file_path>", [](peregrine::Request& req) {
    std::string path = req.path_params["file_path"];
    return peregrine::Response::text("Accessing: " + path);
});
```

10. Request & Response Handling

```
app.post("/submit", [](peregrine::Request& req) {
    // 1. Query parameters (?key=value)
    std::string tab = req.arg("tab", "default_tab");

    // 2. Headers
    std::string auth_header = req.header("Authorization");

    // 3. Form data
    std::string username = req.form_get("username");

    // 4. JSON Payload
    peregrine::Json payload = req.get_json();

    // 5. Uploaded Files
    if (!req.files.empty()) {
        const auto& file = req.files[0];
        file.save("uploads/" + file.filename);
    }

    return peregrine::Response::text("Success");
});
```

11. Real-Time Video & Media Streaming

```
app.get("/mjpeg", [](peregrine::Request&) {
    return peregrine::Response::mjpeg([](peregrine::Connection& conn) {
        while (true) {
            std::string jpeg_frame = capture_camera_frame(); // binary JPEG

            std::string header = "--frame\r\nContent-Type: image/jpeg\r\nContent-Length: " +
                std::to_string(jpeg_frame.size()) + "\r\n\r\n";
            std::string footer = "\r\n";

            if (conn.write(header.data(), header.size()) <= 0) break; // Client disconnected
            if (conn.write(jpeg_frame.data(), jpeg_frame.size()) <= 0) break;
            if (conn.write(footer.data(), footer.size()) <= 0) break;

            std::this_thread::sleep_for(std::chrono::milliseconds(33)); // 30 FPS
        }
    });
});
```

12. Security Architecture

1. **Directory Traversal (CWE-22):** Rejects any static request containing `..` or `\0` null-bytes before file system interaction.
 2. **Timing Attack Resistant Comparisons:** `peregrine::constant_time_equal` used for all session signatures and CSRF tokens.
 3. **Session Tamper Protection:** Signed with `HMAC-SHA256` and Unix expiration timestamps.
 4. **Memory Safety:** Clean modern C++17 value types without manual pointer arithmetic.
 5. **TLS 1.2 / TLS 1.3:** Strong OpenSSL cipher suites enforced in HTTPS mode.
-

13. Compatibility Matrix

Framework Version	C++ Standard	POSIX / Linux	macOS	Windows (MinGW/WSL)	OpenSSL Version
1.0.0	C++17	Verified	Verified	Verified	1.1.1, 3.0.x, 3.1.x, 3.2.x

14. Final Integration Checklist

Before deploying applications built with Peregrine to production, verify:

- ☒ C++17 Compiler & POSIX environment verified
- ☒ OpenSSL runtime headers installed (`libssl-dev`)
- ☒ `app.secret_key` loaded from environment variables in production
- ☒ All state-changing endpoints protected with CSRF or Token Auth
- ☒ Shared in-memory data structures guarded with `std::mutex`
- ☒ Static file directories configured with path-traversal protection
- ☒ HTTPS certificates and private keys verified with `SSL_CTX_check_private_key`
- ☒ Automated integration test suites passing
- ☒ Reverse proxy (Nginx) and systemd service descriptors configured